UQAC
Université du Québec
à Chicoutimi

RAPPORT DE TRAVAIL PRATIQUE

Fondement Théorique des moteurs de jeux vidéo

#1

PAR

Emmanuelle Cloutier CLOE15510400

Anthony Desbiens DESA07010400

Coralie Tremblay TREC09530303

1. RESSOURCES UTILISÉES

Pour réaliser ce projet et comprendre les concepts de ray casting en 2D, nous avons consulté plusieurs sources :

1. ChatGPT - pour mieux comprendre la logique du ray casting, la structure du code en processing et l'implémentation des intersections entre rayons et murs.
2. Youtube - pour visualiser des tutoriels et des démonstrations de ray casting en 2D et comprendre les principes visuels et interactifs
3. Blog de Tim Allan Wheeler - Explication sur la logique de ray casting utilisée dans les jeux classiques de type Wolfenstein 3D
   Wheeler, T. A. (2023). *Wolfenstein 3D Raycasting in C*. Consulté sur https://timallanwheeler.com/blog/2023/04/01/wolfenstein-3d-raycasting-in-c/
4. Stack Overflow - pour des exemples pratiques en Java sur la détection d'intersections entre lignes et la gestion de rayons dans une scène 2D
   Stack Overflow (2017). *Java: Trying to raycast in 2D*. Consulté sur https://stackoverflow.com/questions/45112876/java-trying-to-raycast-in-2d

Ces ressources ont été utilisées pour comprendre la théorie, vérifier la logique du code et s'inspirer de techniques avancées, tout en développant notre propre implémentation sur Processing

2. EXPLICATION DE LA SOLUTION

Dans ce travail pratique, nous avons réalisé un système de ray casting 2D avec Processing en Java. L'objectif est de simuler l'envoi de rayons dans une scène contenant des murs, afin de détecter les points d'intersection entre ces rayons et les obstacles présents.

La solution repose sur une structure simple composée de deux classes principales : une classe représentant les murs et une classe représentant les rayons. Les murs sont définis comme des segments de droite à partir de deux points dans l'espace, tandis que des listes dynamiques pour stocker ces objets, ce qui permet de gérer plusieurs murs et plusieurs rayons simultanément. Au démarrage du programme, plusieurs murs sont générés de façon aléatoire dans la scène. Les rayons sont générés en grand nombre autour d'un point d'origine, avec des angles répartis sur 360 degrés.

Le fonctionnement principal repose sur le calcul des intersections entre chaque rayon et l'ensemble des murs. Pour chaque image affichée, chaque rayon est testé contre tous les murs présents dans la scène. Le calcul d'intersection est basé sur les équations mathématiques des droites, permettant de déterminer si deux segments se croisent et de calculer précisément le point d'intersection. Une vérification est ensuite effectuée pour s'assurer que le point trouvé est bien situé sur le segment du mur et dans la direction du rayon. Lorsqu'un rayon intersecte plusieurs murs, il est important de ne conserver que l'intersection la plus proche de son point d'origine. Il faut donc que la distance entre l'origine du rayon et chaque point d'intersection soit calculée, puis la plus petite distance est retenue.

Au début du code, il y a une liste qui est créée pour stocker les murs et une autre pour les rayons. Dans la fonction setup(), on initialise la fenêtre graphique et ces deux listes. Ensuite, cinq murs sont générés aléatoirement dans l'espace, ce qui crée un environnement variable à chaque exécution. On ajoute aussi quatre murs fixes sur les bords de l'écran pour fermer la scène.

Toujours dans setup(), on crée plusieurs rayons qui partent initialement du centre de l'écran, chacun avec une direction différente. Ces rayons sont représentés par la classe Ray qui contient une position et une direction

Ensuite, pour chaque rayon, sa position est mise à jour pour suivre la souris. Cela donne l'impression que la source des rayons se déplace avec le curseur. Chaque rayon appelle ensuite la fonction cast() pour détecter les intersections avec les murs.

La méthode intersect() calcule si un rayon touche un mur en utilisant des formules mathématiques d'intersection entre deux segments. Si les lignes sont parallèles, il n'y pas d'intersection. Sinon, on calcule un point potentiel et on vérifie s'il est valide. Si oui, ce point est retourné.

La méthode cast() parcourt tous les murs et trouve le point d'intersection le plus proche du rayon. Cela permet de simuler un rayon qui s'arrête au premier obstacle rencontré. Si une intersection est trouvée, une ligne jaune est dessinée entre la position duy rayon et ce point.

Finalement, les résultats sont affichés à l'écran à chaque frame. Les murs sont dessinés sous forme de lignes, tandis que les rayons sont tracés depuis leur point d'origine jusqu'au point d'intersection le plus proche.

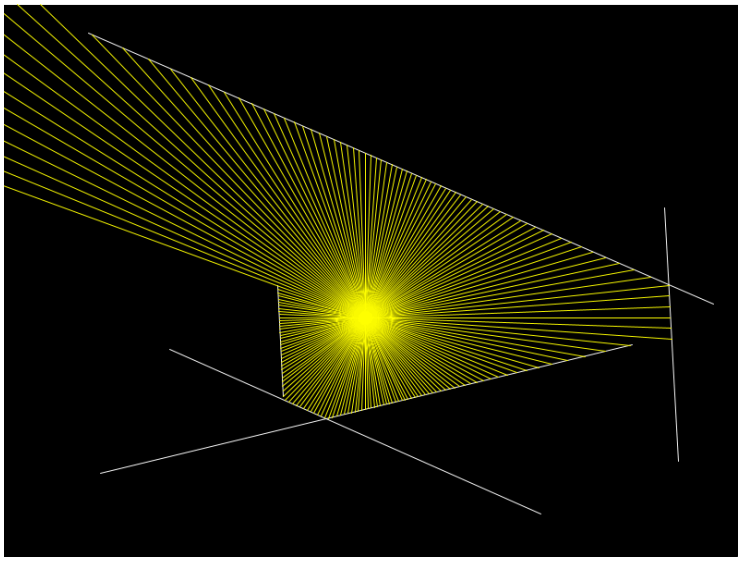

Figure 1 : Programme compilé dans le dossier 2D

3. DISCUSSION

La première erreur que nous avons rencontrée était liée à l'initialisation d'une variable nommée<record>. Processing nous affichait le message d'erreur suivant :

Syntax Error - Error on variable assignment near float record =?

Figure 2 : Message erreur

En fait, record est un mot réservé en Java. La solution a été simple : la renommer.

Un autre problème, signalé par Lucas, apparaissait lorsque la fenêtre n'était pas délimitée par des murs. Certains rayons n'étaient pas générés sur les 360 degrés autour du curseur. Nous avons d'abord pensé à une erreur d'angles, mais nous avons finalement constaté que les rayons ne créaient tout simplement pas lorsqu'ils ne rencontraient aucun murs. Nous avons donc encadré de chaque mur les bords de notre image pour voir tous les rayons.

Le système de ray casting développé détecte correctement les intersections avec les murs en 2D, mais présente des limitations : absence de profondeur, performance réduite avec beaucoup de murs et rendu minimaliste sans effets de lumière.

Plusieurs améliorations sont possibles : limiter les rayons à un cône de vision, ajouter des effets de lumière ou des dégradés, optimiser les calculs pour les grandes scènes avec des grilles ou un quadtree, et même créer une projection pseudo-3D inspirée de Wolfenstein Enfin, des interactions supplémentaires, comme le déplacement du point d'origine des rayons ou des effets visuels, pourraient renforcer l'immersion.

En conclusion, le système fonctionne bien pour le TP, mais ces évolutions offrent un réel potentiel pour un rendu plus immersif et proche des techniques des jeux classiques en ray casting.

4. CONTRIBUTIONS DES MEMBRES DE L'ÉQUIPE

Emmanuelle Cloutier :

- Recherche pour le ray casting 2D et 3D
- Coder dans le modèle 2D du devis
- Rédaction dans le rapport

Anthony Desbiens:

- Débugger dans le modèle 2D du devis
- Rédaction dans le rapport

Coralie Tremblay :

- Recherche pour le raycasting 3D
- Coder dans le modèle 3D du devis

5. DÉPASSE LES ATTENTES

Le devis initial du projet consistait à implémenter un système de ray casting en 2D capable de générer un ensemble de rayons émis à partir d'un point contrôlé par la souris qui permet de détecter les intersections avec des murs positionnés dans la scène.

Dans notre version finale du projet (dossier 3D dans TP1) une amélioration a été apportée en transformant le système de ray casting 2D en rendu style 3D inspiré du jeu Wolfenstein. Cette amélioration dépasse les objectifs initiaux en introduisant une notion de perspective et de profondeur.

Plutôt que d'afficher les rayons directement dans la scène 2D, ceux-ci sont désormais utilisés pour générer une vue en première personne. Chaque rayon correspond à une colonne verticale à l'écran et la distance entre le point d'origine et le mur intercepté est utilisée pour déterminer la hauteur de cette colonne. Ainsi, les murs proches apparaissent plus grands tandis que les murs éloignés apparaissent plus petits, créant une illusion de profondeur.

De plus, le système a été adapté pour simuler un champ de vision limité, contrairement à la diffusion initiale des rayons sur 360 degrés. Cette modification permet de reproduire plus fidèlement la perception visuelle du joueur. Cette approche s'appuie sur les mêmes principes mathématiques que les ray casting initial, mais les exploite de manière plus avancée afin de produire un rendu visuel immersif. Le projet ne se limite donc plus à une simple démonstration technique en 2D, mais devient une simulation simplifiée d'un moteur de rendu 3D.

La version 3D garde la même base que le 2D : on cast des rayons et on trouve la distance au mur le plus proche. Ce qui change, c'est comment on utilise cette distance.

Au lieu d'afficher les rayons à plat, on divise l'écran en colonnes verticales (une par rayon). On réduit aussi le champ de vision à 60 degrés au lieu de 360, pour simuler un vrai champ de vision de joueur.

Pour chaque colonne, la hauteur du mur affiché est calculée avec WallHeight = Scale / distance (plus le mur est proche, plus la colonne est haute).

Un détail important : sans correction, les rayons en bordure du champ de vision semblent plus longs que ceux du centre. Ça déforme l'image et courbe les murs. On corrige ça en multipliant la distance par cos(RayAngle - PlayerAngle), fish-eye.

L'écart entre le devis initial et la version finale illustre une évolution significative du projet. Alors que la version de base visait à démontrer le fonctionnement du ray casting en 2D, la version améliorée en exploite pleinement le potentiel en transformant les données calculées en une représentation visuelle plus complexe.

En conclusion, le projet a évolué d'une implémentation fonctionnelle de base vers une version enrichie intégrant un rendu style 3D. Cette transformation permet non seulement d'améliorer l'aspect visuel, mais aussi de démontrer une maîtrise plus approfondie des concepts abordés. L'ajout d'un système inspiré de Wolfenstein constitue ainsi un dépassement clair des attentes initiales du devis.

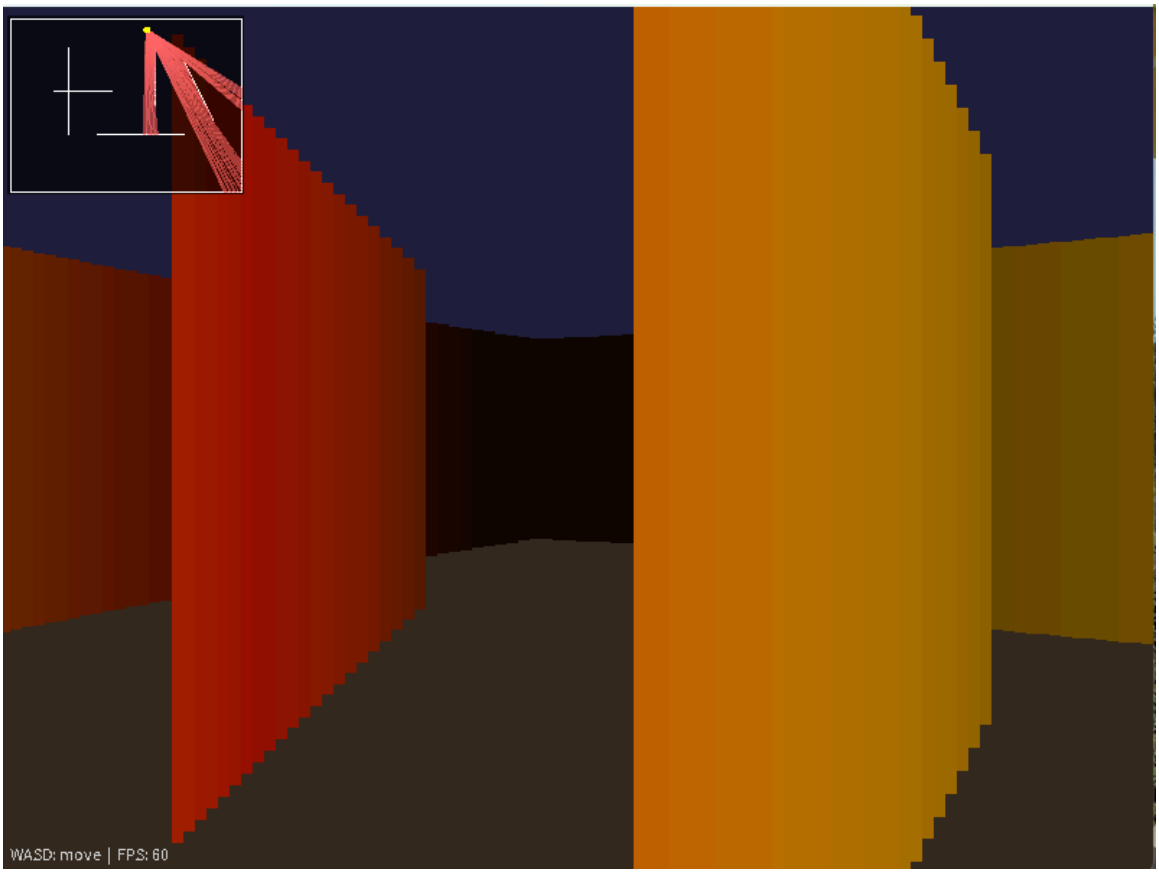

Figure 3 : Programme compilé dans le dossier 3D